FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

KIV/UPP 1. semestrální práce

Analýza vybraných open-data

04.04.2026

Jakub Vokoun

Obsah

1. Zadání **1**
1.1. Popis dat 1
1.2. Zadání práce 1

2. Analýza problému **3**
2.1. Paralelizace úloh 4
2.1.1. Kritická cesta 5
2.2. Datový paralelismus 6
2.3. Teoretické urychlení 7
2.3.1. Amdahlův zákon 7
2.3.2. Gustafsonův zákon 8

3. Implementace paralelizace **8**

4. Experimentálně zjištěné výsledky **9**
4.1. Konfigurace testovacího prostředí 9
4.2. Naměřené časy 9
4.3. Metriky paralelizace 10

5. Závěr **10**

1. Zadání

Zkopírováno z portálu CoureWare.

1.1. Popis dat

Z portletu se soubory si stahněte soubor „SP1 - Průměrné teploty“ - ten bude Vaší výchozí datovou sadou. Pro začátek testujte práci s malou sadou, ta obsahuje původní data. Datové sady označené jako „střední“ a „velké“ byly rozšířeny o stanice náhodně vygenerované.

Původní data jsou extrahována z měření všech meteorologických stanic v ČR. Zdroj dat je ČHMÚ, data jsou volně použitelná dle zákona 123/1998 Sb. Data byla předzpracována a jsou poskytnuta studentům dle oficiálních pokynů. Datové sady větší jsou uměle rozšířeny o náhodné stanice, které přejímají měření nejbližší stanice s vygenerovanou malou odchylkou.

Soubor `stanice.csv` obsahuje seznam měřicích stanic. Jde o CSV soubor s hlavičkou a sloupci oddělenými středníkem, kódování je UTF-8. Obsahuje tyto položky:

- `id` - identifikátor stanice, podle toho budete přiřazovat data z druhého souboru s daty
- `name` - orientační název stanice
- `latitude`, `longitude` - zeměpisná šířka a délka umístění stanice

Soubor `mereni.csv` obsahuje všechna agregovaná měření z těchto stanic. Každý řádek představuje jednu průměrnou denní teplotu z jedné stanice. Jde rovněž o CSV soubor s hlavičkou a sloupci oddělenými středníkem. Obsahuje tyto položky:

- `station_id` - ID stanice ze souboru stanice.csv
- `ordinal` - pořadové číslo měření (doplňkové)
- `year`, `month`, `day` - rok, měsíc a den hodnoty
- `value` - hodnota

Pokud se tedy například objeví řádek `24;125;1994;2;12;3.5`, lze usuzovat, že dle stanice 24 byla 12. února 1994 průměrná teplota v místě měření 3.5 °C (a že to byla 125. hodnota, kterou stanice naměřila).

1.2. Zadání práce

Vaším úkolem bude provést několik transformací a analýzu dat:

1. předzpracujte data
   - odstraňte stanice, které mají naměřeno méně, než 5 let souvislých hodnot (tj. pokud se v datech objevuje kdekoliv alespoň 5 souvislých let, stanici ponechte)

- odstraňte stanice, které za naměřené období reportovaly v průměru méně, než 100 hodnot ročně

2. identifikujte meziročný výkyvy
   - meziroční výkyv je detekován, pokud se průměrná teplota daný měsíc ve dvou po sobě jdoucích letech liší o více, jak 75 % rozdílu mezi minimem a maximem
   - např.: pokud v datech objevíte, že nejvyšší a nejnižší průměrná červnová teplota je 12 a 19 stupňů (tj. ze všech průměrů za všechny roky), pak je hledaná hranice rozdílu 0.75*(19-12) = 5.25 stupně
     - pak když zjistíte, že se průměrná teplota v červnu 2000 a v červnu 2001 liší o 6 stupňů (> 5.25), detekujete výkyv
     - pokud se ale průměrná teplota v červnu 2001 a červnu 2002 liší o 2 stupně, je všechno v pořádku
3. stanovte průměrné teploty ve všech měřicích místech v každém měsíci
4. vykreslete mapu ČR ve formátu SVG pro každý měsíc (leden.svg, …, prosinec.svg), kde vykreslíte všechny stanice jako body, které obarvíte na škále od modré přes žlutou (střed rozsahu) až k červené (lineárně interpolujte)
   - jako minimum škály berte globální minimum teplot ze všech naměřených dat
   - analogicky maximum škály
   - tj. pokud zjistíte, že ze všech dat bylo nejnižší číslo –20, tam bude začínat modrá barva; pokud bude maximem 40, tam bude končit barva červená
   - slepá mapa ČR je k dispozici taktéž v portletu se soubory jako „Slepá mapa ČR (SVG)“
     - jak ji použijete je na Vás - zda ji dáte do kódu jako zdroj a doplníte body nebo ji necháte jako soubor přiloženou, popř. jakákoliv jiná kombinace; důležitý je výstup, ale vyvarujte se pochopitelně očividným nečistým řešením
     - v této mapě levý horní roh odpovídá souřadnicím 51.03806105663445, 12.102209054269062 a pravý spodní roh souřadnicím 48.521003814763994, 18.866923511078615
     - pro vykreslení zanedbejte zakřivení země a umístěte body s lineárně interpolovanými souřadnicemi, jako kdyby byla Země placatá
   - nemusíte implementovat SVG „v plné kráse“ - stačí se naučit, jak vykreslit vyplněné kolečko, které snadno naformátujete jako text
5. do souboru vykyvy.csv vypište identifikované meziroční výkyvy
   - formátem je standardní CSV se středníkem jako oddělovačem
   - zapište hlavičku CSV, budete ukládat následující sloupce:
     - ID stanice
     - měsíc detekce (jeho číslo, tj. 1 = leden, …)
     - druhý z roků detekce (tj. pokud jste detekovali výkyv mezi roky 2000 a 2001, zapište 2001)

    - hodnota rozdílu teplot
6. zdokumentujte Vaše uvažování
   - vytvořte průvodní dokument, ve kterém popíšete, jak jste nad problémem uvažovali
   - proveďte důkladnou analýzu problému, identifikujte vhodná místa pro paralelizaci a následně zhodnoťte implementaci
     - určitě uveďte popis paralelní úlohy (precedenční graf, kritická cesta, vydedukované teoretické urychlení z toho)
   - spočítejte metriky relevantní k paralelizaci
     - urychlení
     - efektivita paralelizace
     - Amdahlův zákon - porovnejte s očekáváním, zdůvodněte výsledky
     - Gustafsonova metrika - porovnejte s očekáváním, zdůvodněte výsledky
   - uveďte, na jakém PC jste k výsledkům došli (CPU, RAM)

Paralelizujte všechny kroky v nejvyšší možné míře, která by se mohla vyplatit. Svá rozhodnutí dobře zdůvodněte.

Program bude spouštěn z příkazové řádky, kdy prvním parametrem je vždy cesta k souboru se stanicemi, druhým parametrem vždy cesta k souboru s odpovídajícím měřením a třetím parametrem je vždy přepínač sériové/paralelní verze. **Řešení tedy obsahuje obě verze - jak sériovou, tak paralelní!** Příklad:

```
./upp_sp1 stanice.csv mereni.csv --serial
```

```
./upp_sp1 C:\Data\stanice.csv ..\mereni.csv --parallel
```

2. Analýza problému

V této kapitole je primárně rozebírán problém paralelizace, tedy ne jak implementovat samotné řešení, ale jak jej paralelizovat a optimalizovat. Na úvod této kapitoly je ale nezbytné zmínit některé důležité skutečnosti.

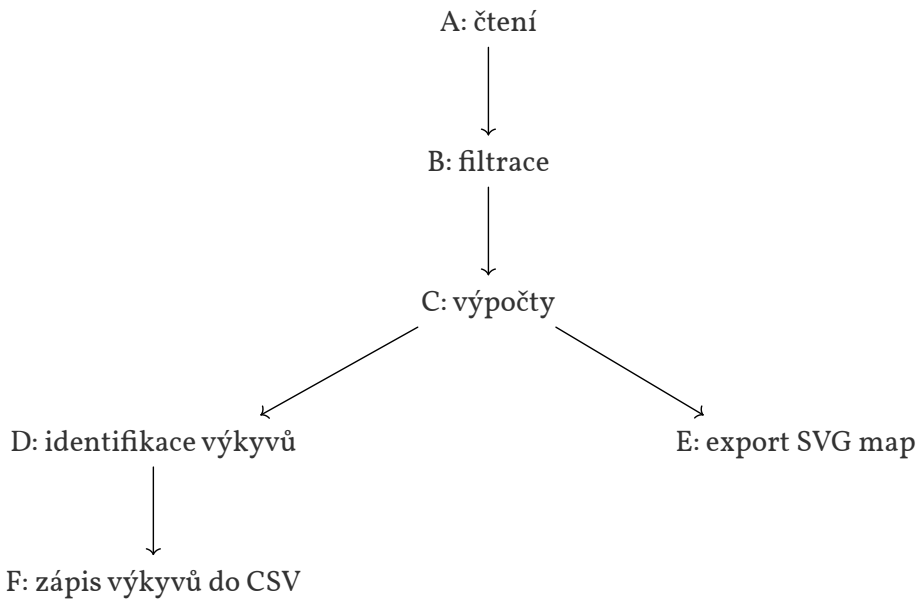
- Řešení všech dílčích problémů bude snažší, pokud jsou data seřazená - stanice podle svých identifikačních čísel uloženy do vektoru, měření podle pořadového čísla. Nebude problém pracovat nad daty takovým způsobem, aby se jim neměnilo pořadí. Je tedy důležité načíst data ve správném pořadí, případně je po načtení seřadit.
  - → Zaveden předpoklad, že načítaná data jsou pěkná - správně seřazená. Tento předpoklad byl experimentálně ověřen pro danou množinu dat. Pro jakákoliv další data by nemusel platit.

2.1. Paralelizace úloh

Úlohu si můžeme rozdělit na podproblémy:

A načtení dat ze souborů
B předzpracování dat (1)
C práci nad daty (3)
D hledání výkyvů (2)
E vytváření SVG map (4)
F zápis do CSV souboru (5)

Přičemž v závorkách je uvedena souvislost s body ze zadání.



Obrázek 1: Precedenční graf úlohy.

Precedenci úloh pozorujeme na grafu Obrázek 1. Graf téměř přesně kopíruje úlohy v zadání, ale vznikl na základě analýzy vstupů a výstupů jednotlivých úloh (zaznamenané v Tabulka 1).

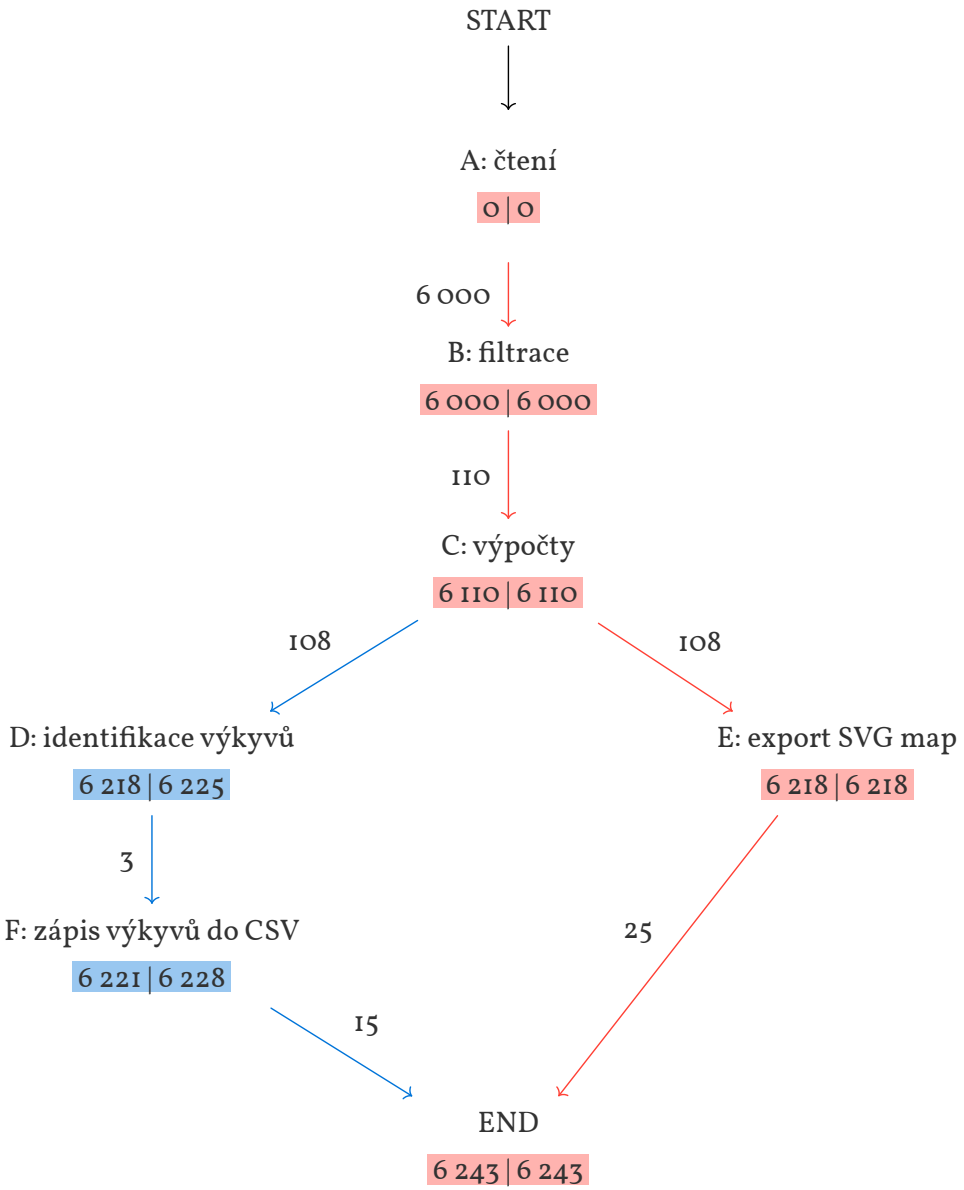
Úloha	Vstup	Výstup
A - Načtení dat	cesty ke vstupním souborům	vektor stanic, každá obsahuje svůj vektor měření
B - Filtrace	vektor všech stanic	vektor obsahující pouze platné stanice
C - Statistiky	vektor stanic	vektor stanic, každá obsahuje spočtené statistiky
D - Výkyvy	vektor stanic obsahující statistiky	vektor stanic, každá obsahuje svůj vektor detekovaných výkyvů
E - SVG mapy	vektor stanic obsahující statistiky	12 obrázků, každý pro jeden měsíc
F - zápis CSV	vektor stanic obsahující detekované výkyvy	1 soubor se všemi detekovanými výkyvy

Tabulka 1: Seznam identifikovaných úloh včetně jejich vstupů a výstupů.

Na základě znalostí uvedených v Obrázek 1 a Tabulka 1 očekáváme, že úloha je alespoň částečně paralelizovatelná na úrovni podúloh, a to ve fázi následující po podproblému C (výpočet statistik).

2.1.1. Kritická cesta

Pro nalezení kritické cesty použijeme standardní algoritmus, jehož provedení je znázorněno v Obrázek 2 (do grafu byly uměle přidány body startu a konce běhu programu). Pro hledání byly použity zprůměrované časy jednotlivých úloh v sériovém řešení (může se lišit od časů v paralelní verzi).



Obrázek 2: Kritická cesta podle časů získaných pomocí sériové verze, časy v milisekundách. Kritická cesta je vyznačená červenou barvou.

2.2. Datový paralelismus

Všechny úlohy (A-F) lze paralelizovat, například pomocí cyklů. Následuje analýza, jakým způsobem paralelizovat konkrétní úlohy spolu s původním sériovým řešením v Tabulka 2.

Úloha	Sériová verze	Paralelní verze
A - Načtení dat	Načtení celého souboru do bufferu, následné procházení bufferu a načítání do struktur.	Po načtení bufferu jej rozdělit podle počtu jader, každé načítá část.
B - Filtrace	Sekvenční kontrola všech stanic pomocí nezávislých funkcí.	Využití trapného paralelismu (funkcí standardní knihovny), rozdělení stanic mezi jádra.
C - Statistiky	Výpočty průměrů, minimálních a maximálních hodnot jsou na sobě mezi stanicemi nezávislé, vykonávají se sekvenčně.	Trapný paralelismus, rozdělení stanic mezi jádra.
D - Výkyvy	Detekce výkyvů na sobě mezi stanicemi nezávislá, vykonává se sekvenčně.	Trapný paralelismus, rozdělení stanic mezi jádra.
E - SVG mapy	Cyklus pro všechny měsíce, každý zakreslí všechny stanice.	Trapný paralelismus, rozdělení měsíců mezi jádra. Nedojde ke konfliktu, protože každé jádro píše do jednoho souboru a ze stanic pouze čte.
F - zápis CSV	Prochází všechny stanice a zapisuje jejich výkyvy do souboru.	Rozdělení stanic mezi jádra, připravují řetězec k zapsání, které proběhne poté na jednom jádře.

Tabulka 2: Shrnutí fungování sériové verze programu a navržení paralelní.

2.3. Teoretické urychlení

Předpokládejme, podle informací v Tabulka 2, že všechny úlohy mohou být paralelizované.

2.3.1. Amdahlův zákon

Amdahlovův zákon definuje urychlení jako:

$$S_{\text{lat}} = \frac{1}{(1-p) + \frac{p}{s}} \tag{1.}$$

kde p je podíl čas běhu paralizovatelných úloh vůči všem úlohám a s je maximální možné urychlení.

Úloha	Čas [ms] (sériová verze)
A - Načtení dat	6 000
B - Filtrace	110
C - Statistiky	108
D - Výkyvy	3
E - SVG mapy	25
F - zápis CSV	15
Celkem	**6 261**

Tabulka 3: Průměrné doby vykonávání jednotlivých úloh sériovým způsobem.

Podle předpokladu, že všechny úlohy jsou paralelizovatelné, z Tabulka 3 vidíme, že $p = \frac{\sum_A^F(t)}{24905} = 1$ a řekneme, že $s = \#\text{jader} = 8$. Po dosazení do rovnice dostáváme:

$$S_{\text{lat}} = \frac{1}{(1-1) + \frac{1}{8}} = 8 \qquad 2.$$

Z Rovnice 2 vidíme, že teoretické maximální zrychlení je 800%. To odpovídá naivnímu předpokladu, že se všechny úlohy podaří rozdělit mezi osm jader.

2.3.2. Gustafsonův zákon

Definován jako:

$$S = (1-a)P + a \qquad 3.$$

kde a je podíl sériové části kódu a P je počet procesorových jader.

Pro náš předpoklad, že je celý program paralelizovatelný, dostáváme $a = 0$.

$$S = (1-0)8 + 0 = 8 \qquad 4.$$

3. Implementace paralelizace

Pro účely paralelizace úlohy A (načítání dat) a F (zápis do CSV) byl vytvořen jednoduchý vláknový rybníček. V úloze A se

1. načetla všechna data do řetězcového bufferu
2. každé vlákno dostalo stejně velkou část bufferu ke zpracování
3. po skončení práce všech vláken se výsledky načtení v hlavním vlákně spojily.

Každé vlákno používá lokální vektor, po dokončení práce všech se vektory propíší do hlavního. Nezměnění pořadí dat je zajištěno tím, že každé vlákno zná index

své práce (např. 3. část z 8) a na tom indexu i ukládá svou práci do bufferovacího vektoru. Poté stačí do hlavního vektoru přepisovat práci ve stejném pořadí, v jakém ji vlákna zpracovaly.

Bylo by mohlo být efektivnější použít k signalizaci semafor pro každé vlákno - tím pádem by hlavní vlákno nečekalo na dokončení všech prací, ale pouze první (pak by proběhl merge), následně druhé atd. Ze dvou důvodů však byla použitá past (latch).

1. jednoduchost/přehlednost kódu
2. každé vlákno má přibližně stejně velkou práci, předpoklad, že ji dokončí v podobném čase

V úloze F se podobným způsobem mezi vlákna rozdělila práce sbírání dat ze stanic. Následně se v sériové části do souboru zapisovaly větší části textu, než pouhé řádky (tak je to v sériové verzi).

Úlohy B, C, D, E jsou všechny trapně paralelní a tedy byly použité funkce `parallel STL`, konkrétně `for_each(std::execution::par, ...)` a v úloze B `std::remove_if(std::execution::par, ...)`. V úlohách B, C, D se paralelizovalo vzhledem ke stanicím (existují funkce pro zpracování jedné stanice a v sériové verzi se akorát tato funkce volala ve smyčce pro všechny stanice), zatímco v úloze E (svg mapy) se paralelizovalo vzhledem k měsícům.

4. Experimentálně zjištěné výsledky

4.1. Konfigurace testovacího prostředí

Referenční měření probíhalo na stroji s následující konfigurací:

- **CPU:** Intel(R) Core(TM) i5-1035G1 CPU @ 1.00GHz, 4 fyzická / 8 logických jader
- **RAM:** 8 GB DDR4
- **OS:** CachyOS (Linux)
- **Překladač:** GCC 15.2.1
  - Release mode (-O3)

4.2. Naměřené časy

Pro měření byla použita velká datová sada (`velke_mereni.csv`).

Úloha	Čas [ms] (S)	Čas [ms] (P)	Urychlení
A - Načtení dat	6 000	3 500	1.7
B - Filtrace	110	69	1.6
C - Statistiky	108	33	3.3
D - Výkyvy	3	1	3
E - SVG mapy	25	18	1.4
F - zápis CSV	15	4	3.8
Celkem	**6 261**	**3 625**	**1.7**

Tabulka 4: Průměrné doby vykonávání jednotlivých úloh.

4.3. Metriky paralelizace

Na základě celkových časů můžeme vypočítat základní metriky s využitím $p = 8$ vláken.

Dosažené urychlení (Speedup):

$$S = \frac{T_s}{T_p} = \frac{6261}{3625} \approx 1.7 \tag{5.}$$

Efektivita (Efficiency):

$$E = \frac{S}{p} = \frac{1.7}{8} \approx 0.21 \quad (21\%) \tag{6.}$$

5. Závěr

Dosažené hodnoty urychlení paralelizací se ani vzdáleně neblíží ideálnímu zrychlení 8×, které očekává Amdhal i Gustafson. Důvodů je více.

Nejdelší podúlohou bylo načítání vstupních souborů, kde bylo naměřené zrychlení ≈ 1.7 (což téměř přesně odpovídá kompletnímu urychlení). Přestože v tomto případě byla rozdělena práce spojená s parsováním CSV souboru mezi osm vláken, režie potřebná pro spojení částečných výsledků nedovolila výraznější zrychlení.

Za povšimnutí stojí fakt, že největších zrychlení jsme dosáhlu u úloh C, D - to není tolik překvapivé, protože obě dvě využívají trapný paralelismus. Zároveň ale absolutně největšího zrychlení dosáhla úloha F, tedy zápis do souboru. Ukázalo se, přestože jsem o tom měl pochybnosti, že režie spojená s vytvářením řetězcových bufferů je málo podstatná oproti sekvenčnímu zápisu do paměti.

Očekával jsem větši zrychlení u úlohy E (generování SVG map). Její špatnou optimalizaci připisuji mezivláknovým přístupům do stejné oblasti paměti. Všechna vlákna prochází všechny stanice, takže si nejspíš vzájemně zneplatňují cache.

Po analýze precedenčního grafu jsem zavedl paralelismus pomocí vlákeného rybníčku i do funkce `parallel_version` v souboru `main.cpp`. Empirickým testováním (průměr ze 100 běhů) jsem ale zjistil, že verze **s rybníčkem** je pomalejší o vyšší desítky milisekund.